NEXT ENGINEER · _SHEERAT_

Web Technology

BCS502

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 5 | **Credits:** 4 | **L-T-P:** 3-1-0

Scheme: AKTU B.Tech 3rd year, 2024-25 onwards

UNIT 1	UNIT 2	UNIT 3	UNIT 4	UNIT 5
Web Basics, HTML and XML	CSS and Advanced CSS	JavaScript, AJAX and Java Networking	Enterprise Java Beans, Node.js and MongoDB	Servlets and JSP

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables and the theory behind each technology.
3. **Solved examples**: small worked programs and calculations with their output.
4. **Code to remember**: the programs most often asked (forms, CSS layouts, JS validation, sockets, Express API, servlets, JSP).
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. Internet protocols, client-server, HTML forms and tables, XML with DTD and XSD, DOM vs SAX; CSS selectors, box model, positioning and layout.
- **Afternoon (3 hours):** Units 3 and 4. JavaScript objects, form validation and AJAX; InetAddress, TCP and UDP socket programs; JavaBeans and EJB types; Node.js, Express and MongoDB CRUD.
- **Evening (2 hours):** Unit 5. Servlet life cycle, GET vs POST, session tracking (cookies, URL rewriting, hidden fields, HttpSession), JSP elements, implicit objects, directives and actions. Then the "Last-hour checklist".
- **In the exam:** every long answer should have a definition, a diagram (architecture or life cycle) and a short program. Programs need not be long; correct syntax matters.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Web Basics, HTML and XML

✍️ 2-mark definitions

- **Internet:** a global network of networks that communicate using the TCP/IP protocol suite.
- **World Wide Web (WWW):** a system of linked hypertext documents and resources on the Internet, accessed with browsers using HTTP. Invented by Tim Berners-Lee at CERN in 1989.
- **Web protocol:** a set of rules for communication on the web, e.g. HTTP, HTTPS, FTP, SMTP, DNS.
- **HTTP:** HyperText Transfer Protocol, a stateless request-response protocol (port 80); HTTPS adds TLS encryption (port 443).
- **URL:** Uniform Resource Locator, the address of a resource: protocol://host:port/path?query#fragment.
- **Client-server computing:** a model where clients send requests and a server processes them and returns responses, e.g. browser and web server.
- **Web development strategy:** the plan for building a site: defining goals and audience, planning content and structure, design, development, testing, deployment and maintenance.
- **HTML:** HyperText Markup Language, the standard tag-based language for structuring web pages.
- **Frame:** a division of the browser window that shows a separate HTML document (frameset is obsolete in HTML5; iframe is still used).
- **XML:** eXtensible Markup Language, a self-describing, text-based format for storing and transporting data with user-defined tags.
- **Well-formed XML:** follows XML syntax rules (one root, properly nested and closed tags, quoted attributes, case-sensitive). **Valid XML** is well-formed and also follows a DTD or schema.
- **DTD:** Document Type Definition, which declares the allowed elements, attributes and structure of an XML document.
- **XML Schema (XSD):** an XML-based, more powerful alternative to DTD that supports data types and namespaces.
- **DOM:** Document Object Model, a tree representation of a document loaded fully in memory.
- **SAX:** Simple API for XML, an event-based parser that reads the document sequentially and fires events.

💡 Core concepts

Common Internet protocols and services

Protocol	Use	Port
HTTP / HTTPS	web pages	80 / 443
FTP	file transfer	20, 21
SMTP	sending e-mail	25

Protocol	Use	Port
POP3 / IMAP	receiving e-mail	110 / 143
DNS	domain name to IP address	53
Telnet / SSH	remote login	23 / 22

Internet services and tools: e-mail, WWW, FTP, chat and messaging, video conferencing, search engines, newsgroups, and remote login. Connecting to the Internet needs an ISP and a connection type (dial-up, DSL, cable, fibre, wireless, mobile data).

HTTP request methods and status codes

- Methods: GET (read), POST (create / submit), PUT (replace), PATCH (partial update), DELETE (remove), HEAD (headers only).
- Status codes: 1xx informational, 2xx success (200 OK, 201 Created), 3xx redirection (301, 302), 4xx client error (400, 401, 403, 404), 5xx server error (500, 503).

HTML lists: ordered (<ol>, <li>), unordered (<ul>) and description (<dl>, <dt>, <dd>) lists.

HTML tables: <table>, <tr>, <th>, <td>, with colspan and rowspan to merge cells, and <caption>, <thead>, <tbody>.

HTML forms: <form action="url" method="get|post"> containing <input> (type text, password, email, number, radio, checkbox, submit, reset, file, date), <select>/<option>, <textarea> and <button>.

HTML vs XML

Point	HTML	XML
Purpose	display data	store and transport data
Tags	predefined	user-defined
Case	not case-sensitive	case-sensitive
Closing tags	some may be omitted	every tag must close
Errors	browsers tolerate	must be well-formed

DTD vs XSD

Point	DTD	XML Schema
Syntax	own (non-XML) syntax	written in XML
Data types	only text (#PCDATA)	many (string, int, date ...)
Namespaces	not supported	supported
Extensible	no	yes

DOM vs SAX

Point	DOM	SAX
Model	tree in memory	events (startElement, characters, endElement)

Point	DOM	SAX
Memory	high (whole document)	low
Access	random, read and modify	sequential, read only
Speed for large files	slow	fast
Use	small documents needing edits	large documents, one pass

Presenting XML: with CSS (link a stylesheet) or XSLT (transform XML to HTML). Using XML: configuration files, data exchange between applications, RSS feeds, web services (SOAP).

🧮 Solved examples

Example 1 (table with merged cells).

```
<table border="1">
  <caption>Time Table</caption>
  <tr><th rowspan="2">Day</th><th colspan="2">Lectures</th></tr>
  <tr><th>10:00</th><th>11:00</th></tr>
  <tr><td>Mon</td><td>DBMS</td><td>DAA</td></tr>
</table>
```

"Day" spans two rows; "Lectures" spans two columns.

Example 2 (XML with internal DTD).

```
<?xml version="1.0"?>
<!DOCTYPE library [
  <!ELEMENT library (book+)>
  <!ELEMENT book (title, author, price)>
  <!ATTLIST book id CDATA #REQUIRED>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
]>
<library>
  <book id="b1"><title>Web Design</title><author>Xavier</author><price>450</price></book>
</library>
```

DTD symbols: + one or more, * zero or more, ? optional, | choice, comma for sequence.

Example 3 (the same structure in XSD).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
      </xs:sequence>
      <xs:attribute name="id" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

⌨️ Code to remember

```
<!DOCTYPE html>
<html>
<head><title>Registration</title></head>
<body>
  <h2>Student Registration</h2>
  <form action="/register" method="post">
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="email"><br>
    Password: <input type="password" name="pwd"><br>
    Gender: <input type="radio" name="g" value="M"> Male
            <input type="radio" name="g" value="F"> Female<br>
    Skills: <input type="checkbox" name="s" value="java"> Java
            <input type="checkbox" name="s" value="web"> Web<br>
    Branch: <select name="branch"><option>CSE</option><option>IT</option></select><br>
    Address: <textarea name="addr" rows="3" cols="30"></textarea><br>
    <input type="submit" value="Register"> <input type="reset">
  </form>
  <ol><li>HTML</li><li>CSS</li></ol>
  <img src="logo.png" alt="Logo" width="120">
  <a href="https://example.com" target="_blank">Visit</a>
</body>
</html>
```

⭐ Important questions

1. Explain the history of the Internet and the Web, web protocols and the client-server model.
2. Explain web development strategies and the steps in writing a web project.
3. Explain HTML lists, tables, frames, images and forms with code.
4. What is XML? Differentiate HTML and XML; write a DTD and an XML schema for a given document.
5. Explain the XML object model; compare DOM and SAX parsers.

UNIT 2 CSS and Advanced CSS

✍️ 2-mark definitions

- **CSS:** Cascading Style Sheets, a language that describes the presentation (colours, fonts, layout) of HTML documents, keeping style separate from content.
- **Ways to add CSS:** inline (style attribute), internal (<style> in the head) and external (<link rel="stylesheet" href="x.css">). Priority: inline > internal/external (whichever comes later) > browser default.
- **Selector:** the part of a rule that chooses elements: element (p), id (#main), class (.note), universal (*), group (h1, h2), descendant (div p), child (div > p), attribute (input[type="text"]).
- **Cascade and specificity:** when rules conflict, the more specific selector wins: inline (1000) > id (100) > class / attribute / pseudo-class (10) > element (1); later rules win ties; !important overrides.
- **Box model:** every element is a box of content, padding, border and margin, from inside to outside.
- **Block vs inline element:** block elements start on a new line and take full width (div, p, h1); inline elements flow within a line and take only needed width (span, a, img).
- **Pseudo-class:** a keyword for a special state, e.g. :hover, :visited, :first-child, :nth-child(2).
- **Pseudo-element:** styles part of an element, e.g. ::first-letter, ::before, ::after.
- **Image sprite:** many small images combined into one file; background-position shows the needed part, reducing HTTP requests.
- **Float:** moves an element to the left or right so text wraps around it; clear stops the wrapping.

💡 Core concepts

Position property

Value	Behaviour
static	default; normal flow; top/left ignored
relative	offset from its normal position; space is kept
absolute	removed from flow; placed relative to the nearest positioned ancestor
fixed	placed relative to the browser window; stays when scrolling
sticky	relative until a scroll offset is reached, then fixed

Display property: block, inline, inline-block (inline but accepts width and height), none (hidden and takes no space), flex and grid (layout). visibility: hidden hides an element but keeps its space.

Box sizing: with content-box (default) the total width = width + left and right padding + borders + margins; with border-box, width already includes padding and border.

Styling text and fonts: color, text-align, text-decoration, text-transform, line-height, letter-spacing; font-family, font-size, font-weight, font-style.

Background properties: background-color, background-image, background-repeat, background-position, background-size, background-attachment.

Lists and tables: list-style-type (disc, circle, square, decimal), list-style-image; border-collapse, padding in cells, nth-child(even) for striped rows.

Colour values: names (red), hex (#ff0000), rgb(255, 0, 0), rgba with transparency, hsl.

Page layout and site design: classic layouts use a header, navigation bar, main content with sidebar and footer; built with float, flexbox (one-dimensional) or grid (two-dimensional). Good site design follows consistency, readability, navigation clarity, responsiveness (media queries) and fast loading.

🧮 Solved examples

Example 1 (box model width). div { width: 200px; padding: 10px; border: 5px solid; margin: 20px; }

- Content-box: total width = 200 + 2·10 + 2·5 + 2·20 = **270 px** (250 px without margins).
- With box-sizing: border-box: the visible box is 200 px; total with margins = **240 px**.

Example 2 (specificity). Rules p { color: blue } (0-0-1), .note { color: green } (0-1-0), #intro { color: red } (1-0-0). For <p id="intro" class="note">, the text is **red** because the id selector is most specific.

Example 3 (horizontal navigation bar).

```
ul.nav { list-style: none; margin: 0; padding: 0; background: #333; overflow: hidden; }
ul.nav li { float: left; }
ul.nav li a { display: block; color: white; padding: 14px 16px; text-decoration: none; }
ul.nav li a:hover { background: #4f46e5; }
```

Example 4 (image sprite). One file icons.png holds a home icon at (0, 0) and a mail icon at (−40px, 0):

```
.icon { width: 40px; height: 40px; background: url(icons.png) no-repeat; }
.home { background-position: 0 0; }
.mail { background-position: -40px 0; }
```

⌨️ Code to remember

```
/* Two-column page layout with flexbox and a responsive rule */
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; }
header, footer { background: #1e293b; color: #fff; padding: 16px; text-align: center; }
.container { display: flex; }
nav  { flex: 1; background: #f1f5f9; padding: 12px; }
main { flex: 3; padding: 12px; }
a:link { color: #4f46e5; } a:visited { color: purple; } a:hover { text-decoration: underline; }
input[type="text"] { border: 1px solid #94a3b8; padding: 6px; }
table { border-collapse: collapse; } td, th { border: 1px solid #ccc; padding: 6px; }
tr:nth-child(even) { background: #f8fafc; }
.box { position: relative; } .badge { position: absolute; top: 0; right: 0; }
@media (max-width: 600px) { .container { flex-direction: column; } }
```

⭐ Important questions

1. What is CSS? Explain the three ways of adding CSS and the cascading order with examples.
2. Explain CSS selectors (id, class, grouping, attribute, pseudo-class) and specificity.
3. Explain the CSS box model with a diagram and a width calculation.
4. Explain display, positioning and floating with examples; build a navigation bar.
5. Explain image sprites, page layout and principles of site design.

NEXT ENGINEER _sheerat_

UNIT 3 JavaScript, AJAX and Java Networking

✍️ 2-mark definitions

- **JavaScript:** a lightweight, interpreted, object-based scripting language that runs in the browser (and on servers with Node.js) to make pages interactive.
- **Document object:** the object that represents the loaded HTML page; methods such as getElementById and querySelector access its elements.
- **Event:** an action the browser detects, e.g. onclick, onload, onsubmit, onchange, onmouseover; a handler function runs in response.
- **var, let, const:** var is function-scoped and hoisted; let and const are block-scoped; const cannot be reassigned.
- **== vs ===:** == compares after type conversion ("5" == 5 is true); === compares value and type ("5" === 5 is false).
- **Built-in objects:** String, Number, Math, Date, Array, RegExp, JSON.
- **AJAX:** Asynchronous JavaScript and XML, a technique for exchanging data with the server in the background (XMLHttpRequest or fetch) and updating part of a page without reloading.
- **IP address:** a numeric address for a host on a network (IPv4 32-bit, IPv6 128-bit).
- **Port:** a 16-bit number (0 to 65535) that identifies a process or service on a host.
- **Socket:** one endpoint of a two-way communication link, identified by IP address and port.
- **InetAddress:** a Java class representing an IP address; created with factory methods since it has no public constructor.
- **URL and URLConnection:** URL represents a web address and its parts; URLConnection represents a connection to it, used to read headers and content.
- **Datagram:** an independent, self-contained message sent over UDP without a connection or delivery guarantee.

💡 Core concepts

JavaScript statements and functions: if/else, switch, for, while, do-while, for...of, for...in, break, continue. Functions can be declared (function f(){}), stored as expressions (const f = function(){}), or written as arrow functions (const f = (a, b) ⇒ a + b).

JavaScript objects: created with literals { name: "Asha", age: 20 }, constructors (new Person()) or classes. Properties are accessed with obj.prop or obj["prop"].

AJAX working:

1. An event occurs on the page.
2. JavaScript creates an XMLHttpRequest object.
3. It sends a request to the server (open and send).
4. The server processes it and returns data (text, JSON or XML).
5. The onreadystatechange / onload handler reads the response.
6. JavaScript updates the page (DOM) without reloading.

readyState values: 0 unsent, 1 opened, 2 headers received, 3 loading, 4 done. status 200 means OK.

InetAddress methods

Factory (static) methods	Instance methods
getLocalHost()	getHostName()
getByName(String host)	getHostAddress()
getAllByName(String host)	getAddress() (byte array)
getByAddress(byte[] addr)	isReachable(int timeout), isMulticastAddress()

TCP vs UDP

Point	TCP	UDP
Connection	connection-oriented	connectionless
Reliability	guaranteed, ordered	not guaranteed
Speed	slower	faster
Java classes	Socket, ServerSocket	DatagramSocket, DatagramPacket
Use	web, e-mail, file transfer	video streaming, DNS, games

URL class methods: getProtocol(), getHost(), getPort(), getPath(), getFile(), getQuery(), openConnection(), openStream().

🧮 Solved examples

Example 1 (JavaScript output).

```
console.log(2 + "3");      // "23"  (number converted to string)
console.log("6" - 2);      // 4     (string converted to number)
console.log(1 == "1");     // true
console.log(1 === "1");    // false
let a = [1, 2, 3].map(x => x * 2);
console.log(a);            // [2, 4, 6]
```

Example 2 (URL parts in Java).

```
URL u = new URL("https://www.example.com:8080/docs/index.html?id=5");
System.out.println(u.getProtocol());  // https
System.out.println(u.getHost());      // www.example.com
System.out.println(u.getPort());      // 8080
System.out.println(u.getPath());      // /docs/index.html
System.out.println(u.getQuery());     // id=5
```

Example 3 (InetAddress).

```
InetAddress local = InetAddress.getLocalHost();
System.out.println(local.getHostName() + " " + local.getHostAddress());
InetAddress[] all = InetAddress.getAllByName("www.google.com");
for (InetAddress a : all) System.out.println(a);
```

⌨️ Code to remember

```
<!-- Form validation with JavaScript -->
<form name="f" onsubmit="return validate()">
  Name: <input type="text" id="name">
  Email: <input type="text" id="email">
  <input type="submit">
</form>
<p id="msg"></p>
<script>
function validate() {
  const name = document.getElementById("name").value.trim();
  const email = document.getElementById("email").value;
  const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (name === "") { document.getElementById("msg").innerHTML = "Name is required"; return false;
}
  if (!pattern.test(email)) { document.getElementById("msg").innerHTML = "Invalid email"; return
false; }
  return true;
}
</script>
```

```
// AJAX with XMLHttpRequest
function loadData() {
  const xhr = new XMLHttpRequest();
  xhr.onreadystatechange = function () {
    if (xhr.readyState === 4 && xhr.status === 200) {
      document.getElementById("out").innerHTML = xhr.responseText;
    }
  };
  xhr.open("GET", "data.txt", true);   // true means asynchronous
  xhr.send();
}
// Modern form: fetch("data.json").then(r => r.json()).then(d => console.log(d));
```

```
// TCP echo server and client
import java.io.*; import java.net.*;

class Server {
    public static void main(String[] a) throws IOException {
        try (ServerSocket ss = new ServerSocket(5000);
             Socket s = ss.accept();                       // waits for a client
             BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
             PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
            String msg = in.readLine();
            out.println("Echo: " + msg);
        }
    }
}

class Client {
    public static void main(String[] a) throws IOException {
        try (Socket s = new Socket("localhost", 5000);
             PrintWriter out = new PrintWriter(s.getOutputStream(), true);
             BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()))) {
            out.println("Hello");
            System.out.println(in.readLine());           // Echo: Hello
        }
    }
}
```

```
// UDP: sender and receiver
DatagramSocket ds = new DatagramSocket();
byte[] data = "Hi".getBytes();
ds.send(new DatagramPacket(data, data.length, InetAddress.getByName("localhost"), 6000));

DatagramSocket rs = new DatagramSocket(6000);
byte[] buf = new byte[1024];
DatagramPacket p = new DatagramPacket(buf, buf.length);
rs.receive(p);
System.out.println(new String(p.getData(), 0, p.getLength()));   // Hi
```

⭐ Important questions

1. Explain JavaScript data types, operators, statements, functions and objects with examples.
2. Write a JavaScript program to validate a registration form.
3. What is AJAX? Explain its working with a diagram and the XMLHttpRequest object.
4. Explain InetAddress with factory and instance methods; explain URL and URLConnection.
5. Write TCP client-server and UDP datagram programs in Java; compare TCP and UDP.

UNIT 4 Enterprise Java Beans, Node.js and MongoDB

✍️ 2-mark definitions

- **JavaBean:** a reusable Java class that is public, has a public no-argument constructor, private properties with getter and setter methods, and is usually Serializable.
- **Bean property types:** simple (one value), indexed (array), bound (notifies listeners on change) and constrained (listeners may veto a change).
- **Enterprise JavaBean (EJB):** a server-side component that runs in an EJB container and holds business logic; the container provides transactions, security, pooling and life-cycle management.
- **Session bean:** an EJB that performs tasks for a client. Stateless beans keep no client data between calls; stateful beans keep conversational state for one client; singleton beans have one shared instance.
- **Entity bean:** an EJB (EJB 2.x) that represents persistent data in a database; replaced by JPA entities (@Entity) in EJB 3.
- **Message-driven bean:** an EJB that processes asynchronous JMS messages.
- **Node.js:** an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine, with an event-driven, non-blocking I/O model, used for server-side programs.
- **REPL:** Read-Eval-Print-Loop, the interactive Node.js shell started with the node command.
- **NPM:** Node Package Manager, which installs and manages packages listed in package.json (npm init, npm install express).
- **Callback:** a function passed to another function and called when an asynchronous task completes.
- **EventEmitter:** the Node.js class (events module) used to create, emit and listen for custom events.
- **Express:** a minimal, flexible Node.js web framework for routing, middleware and building REST APIs.
- **RESTful API:** an API that exposes resources at URLs and uses HTTP methods (GET, POST, PUT, DELETE) on them, statelessly, usually with JSON.
- **MongoDB:** a NoSQL document database that stores data as BSON (JSON-like) documents in collections inside databases.

💡 Core concepts

Stateless vs stateful session bean

Point	Stateless	Stateful
Client state	not kept	kept for one client across calls
Instances	pooled, shared	one per client
Annotation	@Stateless	@Stateful

Point	Stateless	Stateful
Scalability	high	lower
Example	currency converter	shopping cart

EJB types summary: session beans (stateless, stateful, singleton), message-driven beans, and entity beans (older; now JPA entities).

Node.js architecture: a single-threaded event loop handles requests; blocking work (file, network, database) is handed to the libuv thread pool or OS, and a callback is queued when it finishes. This makes Node.js efficient for many concurrent I/O-bound connections but not for CPU-heavy tasks.

Blocking vs non-blocking: fs.readFileSync blocks until the file is read; fs.readFile takes a callback and lets other code run meanwhile.

Packaging: a Node project has package.json (name, version, scripts, dependencies). Modules are shared with module.exports / require (CommonJS) or export / import (ES modules). npm publish shares a package.

SQL vs MongoDB terms

SQL	MongoDB
database	database
table	collection
row	document
column	field
join	$lookup (aggregation) or embedded documents
primary key	_id

MongoDB operations (Node.js driver): insertOne / insertMany, find / findOne with filters ({ age: { $gt: 20 } }), updateOne / updateMany with $set, deleteOne / deleteMany, sort({ field: 1 or -1 }), limit, and aggregate with $lookup for joins. A database or collection is created automatically on the first insert.

🧮 Solved examples

Example 1 (callback order).

```
const fs = require("fs");
console.log("Start");
fs.readFile("a.txt", "utf8", (err, data) => console.log("File read"));
console.log("End");
// Output: Start, End, File read   (the read finishes later, without blocking)
```

Example 2 (EventEmitter).

```
const EventEmitter = require("events");
const shop = new EventEmitter();
shop.on("order", (item) => console.log("Order placed for " + item));
shop.emit("order", "book");   // Order placed for book
```

Example 3 (JavaBean).

```
public class Student implements java.io.Serializable {
    private String name; private int roll;
    public Student() { }
    public String getName() { return name; }  public void setName(String n) { name = n; }
    public int getRoll() { return roll; }      public void setRoll(int r) { roll = r; }
}
```

Example 4 (stateless session bean).

```
@Stateless
public class CalculatorBean {
    public int add(int a, int b) { return a + b; }
}
// Injected in a servlet or another bean with: @EJB CalculatorBean calc;
```

NEXT ENGINEER _sheerat_

⌨️ Code to remember

```
// Express REST API backed by MongoDB
// npm init -y && npm install express mongodb
const express = require("express");
const { MongoClient, ObjectId } = require("mongodb");

const app = express();
app.use(express.json());
const client = new MongoClient("mongodb://localhost:27017");
let students;

app.get("/students", async (req, res) => {
  res.json(await students.find().sort({ name: 1 }).toArray());
});
app.get("/students/:id", async (req, res) => {
  const s = await students.findOne({ _id: new ObjectId(req.params.id) });
  s ? res.json(s) : res.status(404).send("Not found");
});
app.post("/students", async (req, res) => {
  const r = await students.insertOne(req.body);
  res.status(201).json({ id: r.insertedId });
});
app.put("/students/:id", async (req, res) => {
  await students.updateOne({ _id: new ObjectId(req.params.id) }, { $set: req.body });
  res.send("Updated");
});
app.delete("/students/:id", async (req, res) => {
  await students.deleteOne({ _id: new ObjectId(req.params.id) });
  res.send("Deleted");
});

client.connect().then(() => {
  students = client.db("college").collection("students");   // created on first insert
  app.listen(3000, () => console.log("Server on port 3000"));
});
```

```
// MongoDB "join" with $lookup: each student with their department document
db.students.aggregate([
  { $lookup: { from: "departments", localField: "deptId", foreignField: "_id", as: "dept" } }
]);
```

⭐ Important questions

1. What is a JavaBean? Explain bean properties and how to create a bean.
2. Explain EJB architecture and types; compare stateless and stateful session beans; explain entity beans.
3. Explain Node.js: features, event loop, REPL, NPM, callbacks and events.
4. Build a RESTful API using Express; explain routing and middleware.
5. Explain how to connect Node.js to MongoDB and perform create, insert, query, update, delete, sort and join.

UNIT 5 Servlets and JSP

✍️ 2-mark definitions

- **Servlet:** a Java class that runs on a web server (servlet container such as Tomcat), receives HTTP requests and builds dynamic responses.
- **Servlet container:** the part of a web server that loads servlets, manages their life cycle and maps URLs to them.
- **Servlet interface:** the core interface with init(ServletConfig), service(ServletRequest, ServletResponse), destroy(), getServletConfig() and getServletInfo(). GenericServlet implements it; HttpServlet extends GenericServlet.
- **ServletConfig vs ServletContext:** ServletConfig holds initialization parameters for one servlet; ServletContext is shared by all servlets of a web application.
- **Session tracking:** maintaining state across multiple requests from the same user, because HTTP is stateless.
- **Cookie:** a small name-value text stored by the browser and sent back with each request to the same server.
- **HttpSession:** a server-side object that stores user data across requests, identified by a session ID (JSESSIONID).
- **sendRedirect vs forward:** response.sendRedirect() makes the browser issue a new request (URL changes, works across servers); RequestDispatcher.forward() passes the same request to another resource on the server (URL does not change).
- **JSP:** JavaServer Pages, HTML pages with embedded Java that are translated into servlets by the container.
- **JSP implicit objects:** nine objects available without declaration: request, response, out, session, application, config, pageContext, page and exception.
- **Directive:** a message to the JSP container about translating the page: page, include and taglib.
- **Standard action:** an XML-style tag that controls run-time behaviour, e.g. jsp:include, jsp:forward, jsp:useBean, jsp:setProperty, jsp:getProperty, jsp:param.
- **Custom tag library:** user-defined tags implemented by tag handler classes and described in a TLD file, used through the taglib directive.

💡 Core concepts

Servlet architecture: browser → HTTP request → web server → servlet container → (creates request and response objects, finds the servlet from web.xml or @WebServlet) → service() → doGet()/doPost() → response → browser.

Servlet life cycle

1. Loading and instantiation: the container loads the class and creates one instance.
2. init(): called once, for one-time setup.
3. service(): called for every request (in a new thread); it calls doGet, doPost, etc.

4. destroy(): called once before the servlet is removed, to release resources.

GET vs POST

Point	GET	POST
Data sent in	URL query string	request body
Visible	yes	no
Length limit	limited by URL length	no practical limit
Bookmark / cache	possible	not possible
Idempotent	yes	no
Use	fetch data, search	submit forms, upload, login

Session tracking techniques

Technique	How it works	Drawback
Cookies	data stored in the browser	user may disable cookies
Hidden form fields	<input type="hidden"> carries data	works only with forms
URL rewriting	append data or session ID to URLs	visible, every link must be rewritten
HttpSession	server stores data, ID in a cookie or URL	uses server memory

JSP life cycle: translation (JSP → servlet .java) → compilation (.class) → loading and instantiation → jspInit() → _jspService() for each request → jspDestroy().

JSP scripting elements

Element	Syntax	Purpose
Scriptlet	<% code %>	Java statements inside _jspService()
Expression	<%= expr %>	value written to the output
Declaration	<%! decl %>	fields and methods of the servlet class
Comment	<%-- text --%>	not sent to the client
Directive	<%@ ... %>	page settings, include, taglib

Directives: page (import, contentType, session, errorPage, isErrorPage), include (static include at translation time) and taglib (uri and prefix of a tag library).

include directive vs jsp:include action: the directive copies the file at translation time (static); the action includes the output at request time (dynamic).

Servlet vs JSP

Point	Servlet	JSP
Nature	Java code with HTML in strings	HTML with Java inside
Best for	control logic (controller)	presentation (view)

Point	Servlet	JSP
Compilation	compiled by developer	translated and compiled by container
Implicit objects	no	yes
Changes	recompile and redeploy	picked up automatically

🧮 Solved examples

Example 1 (life-cycle output). A servlet printing in each method, requested three times, logs: init once, service three times, then destroy once when the server stops.

Example 2 (cookies).

```
// set
Cookie c = new Cookie("user", "asha");
c.setMaxAge(60 * 60);           // 1 hour
response.addCookie(c);
// read
for (Cookie k : request.getCookies())
    if (k.getName().equals("user")) out.println("Welcome " + k.getValue());
```

Example 3 (HttpSession).

```
HttpSession s = request.getSession();          // creates if absent
s.setAttribute("cart", 3);
int items = (Integer) s.getAttribute("cart");
s.setMaxInactiveInterval(600);                 // 10 minutes
s.invalidate();                                // logout
```

Example 4 (JSP with a bean).

```
<jsp:useBean id="st" class="com.college.Student" scope="session"/>
<jsp:setProperty name="st" property="*"/>
Name: <jsp:getProperty name="st" property="name"/>
```

property="*" copies all matching request parameters into the bean.

⌨️ Code to remember

```
// Login servlet handling GET and POST, with redirect
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    public void init() { System.out.println("init called"); }

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post'>User <input name='u'> "
                + "Pass <input type='password' name='p'> <input type='submit'></form>");
    }

    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String u = req.getParameter("u"), p = req.getParameter("p");
        if ("admin".equals(u) && "1234".equals(p)) {
            req.getSession().setAttribute("user", u);
            res.sendRedirect("welcome.jsp");
        } else {
            res.getWriter().println("Invalid login");
        }
    }

    public void destroy() { System.out.println("destroy called"); }
}
```

```
<%-- welcome.jsp --%>
<%@ page contentType="text/html" import="java.util.Date" %>
<%! int visits = 0; %>
<html><body>
  <% visits++; %>
  <h3>Welcome <%= session.getAttribute("user") %></h3>
  <p>Time: <%= new Date() %>, page visits: <%= visits %></p>
  <jsp:include page="footer.jsp"/>
</body></html>
```

```
// Custom tag: <my:hello name="Asha"/>
public class HelloTag extends SimpleTagSupport {
    private String name;
    public void setName(String name) { this.name = name; }
    public void doTag() throws JspException, IOException {
        getJspContext().getOut().write("Hello, " + name);
    }
}
// Declared in a .tld file; used with <%@ taglib uri="/WEB-INF/my.tld" prefix="my" %>
```

⭐ Important questions

1. Explain servlet architecture and life cycle with a diagram; compare GenericServlet and HttpServlet.
2. Write a servlet to handle GET and POST requests; explain redirection and request forwarding.
3. Explain session tracking techniques; write programs using cookies and HttpSession.

4. Explain JSP life cycle, scripting elements, implicit objects, directives and standard actions with examples.
5. Compare servlets and JSP; explain custom tag libraries with an example.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Protocol and port table; HTTP methods and status codes; client-server model.
- ☐ HTML form tags; table rowspan and colspan; lists.
- ☐ XML well-formed vs valid; DTD symbols; DTD vs XSD; DOM vs SAX.
- ☐ CSS three ways and priority; selectors and specificity; box model 270 px example.
- ☐ position values; display values; float and clear; sprites; navigation bar CSS.
- ☐ JS == vs ===; form validation; AJAX six steps and readyState 4 / status 200.
- ☐ InetAddress factory vs instance methods; TCP vs UDP; Socket/ServerSocket program.
- ☐ JavaBean rules; stateless vs stateful session bean; entity and message-driven beans.
- ☐ Node.js event loop; REPL; NPM; callback order; EventEmitter; Express CRUD routes.
- ☐ MongoDB terms vs SQL; insertOne, find, updateOne with $set, deleteOne, sort, $lookup.
- ☐ Servlet life cycle (init, service, destroy); GET vs POST; sendRedirect vs forward.
- ☐ Session tracking 4 ways; JSP elements, 9 implicit objects, 3 directives, standard actions.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Collaborative Web Development	Jessica Burdman	Addison Wesley	WorldCat
2	Web Technology and Design	C. Xavier	New Age International	WorldCat
3	HTML, DHTML, JavaScript, Perl & CGI	Ivan Bayross	BPB Publications	WorldCat
4	JavaServer Pages	Hans Bergsten	SPD / O'Reilly	WorldCat
5	Internet: The Complete Reference	Margaret Levine Young	McGraw Hill	WorldCat
6	Beginning Node.js, Express & MongoDB Development	Greg Lim	Self-published (2020)	WorldCat
7	MongoDB: The Definitive Guide (3rd ed.)	Shannon Bradshaw, Eoin Brazil, Kristina Chodorow	O'Reilly Media	WorldCat

MDN Web Docs (developer.mozilla.org) is a free, legal companion for Units 1 to 3, and nodejs.org/learn for Unit 4.

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER
sheerat